

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

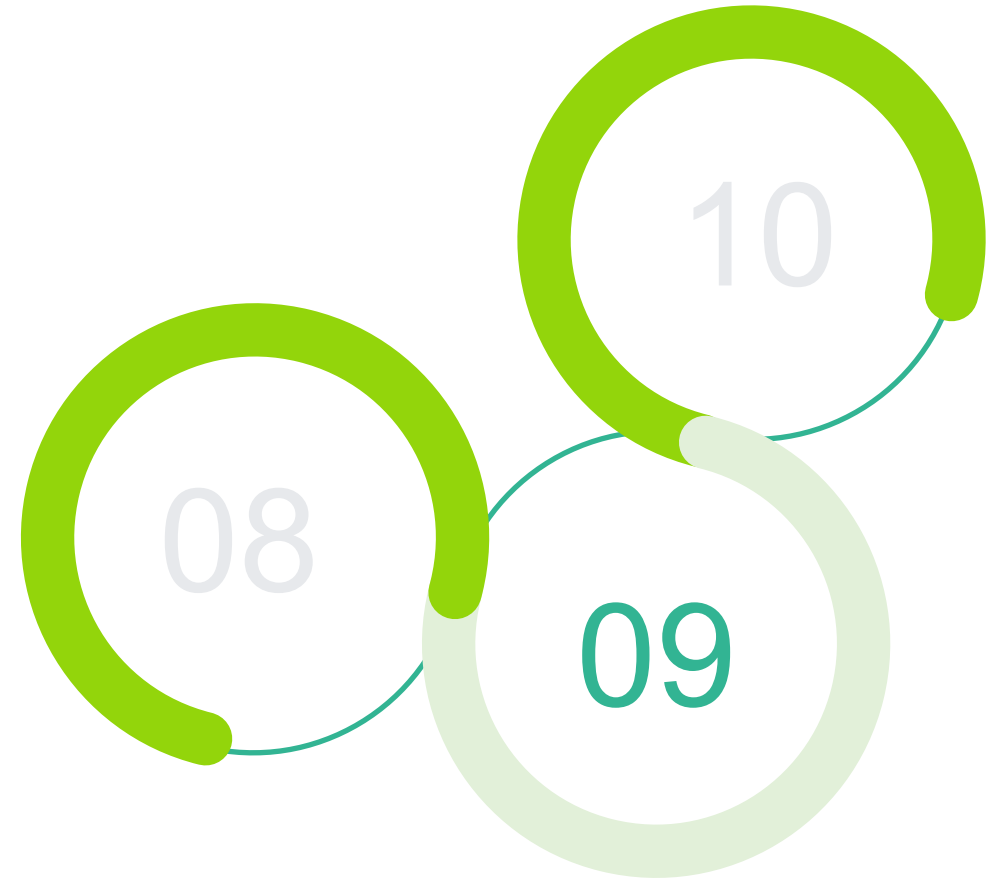
Legacy code management and evolution

Software Engineering
2025/2026



Summary

- Legacy code
- Refactoring techniques
- Black boxes to modern blueprints
- GenAI in Software Engineering



Legacy code

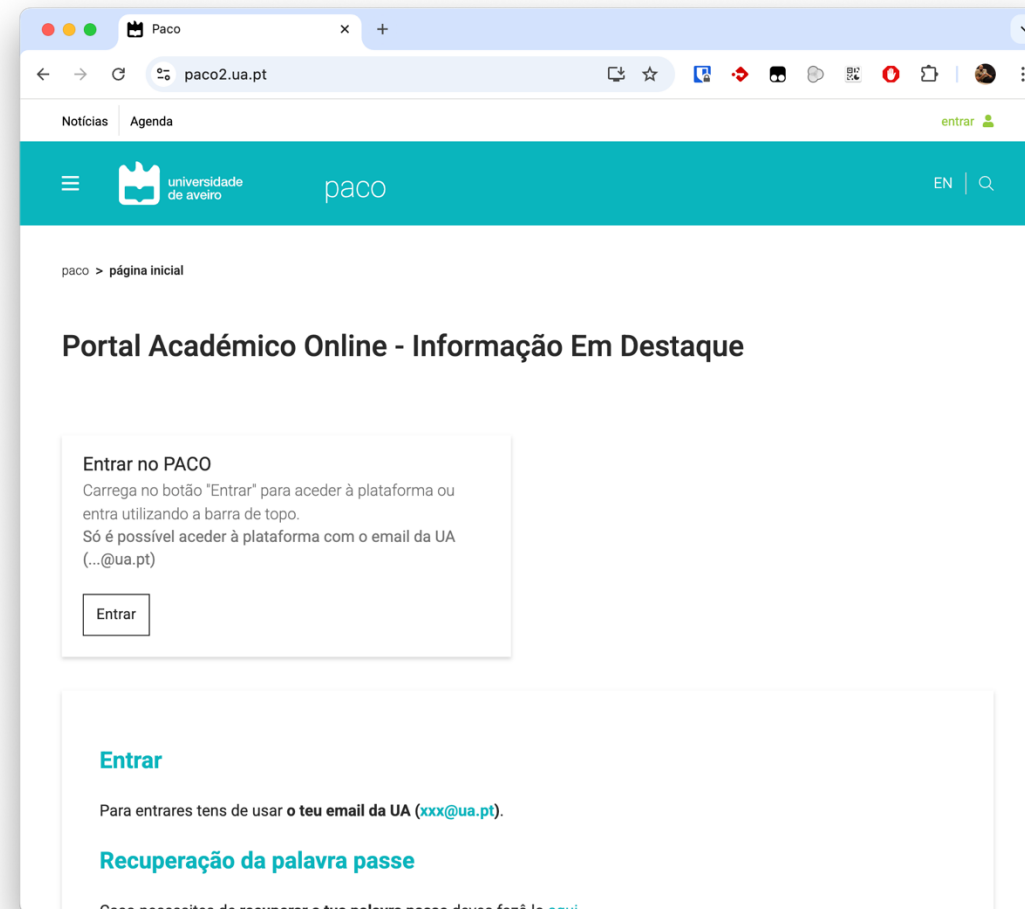
Reality

- The turn of the Millennium
 - A new online academic portal
- One of the first virtual administrative system
 - Students
 - Professors
 - Academic services
- Developed using “novel” technologies — novel back in 2000.



Legacy code

Goal



Legacy code

Definition

- **Legacy Code is code you are afraid to change.**
 - Forget the book definition. It's not about age.
- **Why are you afraid?**
 - Because there are no tests. Full stop.
- **If you can change something and be 100% confident (via automated tests) that you broke nothing, it's not legacy.**
 - Even if it's 20 years old.
- **If you wrote code last week with no tests... congratulations, you wrote "future legacy."**

Legacy code

Anatomy of the mess

- It wasn't malice. Nobody wakes up thinking, "I'm going to write garbage code today."
 - It was pressure. "We need it by Friday!", "No time for tests!", "That's 'gold plating'!"
 - This is called Technical Debt.

- The Analogy (that works):
 - Tech Debt is a payday loan.
 - It gets you the "money" (feature) now
 - but the "interest" (bugs, slowness, frustration) will kill you later.

- We're the debt collectors. We're here to pay it down.

Legacy code

The "Debt Spiral"

1. Pressure to deliver
2. Cut corners (add debt)
3. Code gets harder to change
4. Next feature is slower
5. Pressure increases
6. Cut more corners (more debt)

- (Repeat until you're us, in this room, having this meeting).



Legacy code

The warning signs

- "This function is 2000 lines long." (Complexity)
- "Only João touches this, and he's on vacation." (Tribal Knowledge)
- "To change this label, I have to deploy 3 services." (Coupling)
- "I fixed this bug and three others popped up." (Regressions / Fragile Code)
- Our job is to spot this before we start editing.

Refactoring techniques

Replacing the monster, one piece at a time

➤ Starts at the feature level

- There is some observable behaviour that you can write a pinning test against

➤ A pinning test is a very coarse test.

- It may test a single behaviour that takes hundreds or thousands of lines of code to produce.
- Then as you make changes to the code, you rerun the pinning test to verify that the end-to-end behaviour is still correct.

➤ It effectively decouples objects from the services they use

- To mock those services so code can be tested in isolation.
- This makes smaller units of behavior testable and adds finer-grain unit tests to support more complex refactoring.

Refactoring techniques

Replacing the monster, one piece at a time

➤ There are two industry-proven patterns for this.

- Your choice depends on the scope of the change.

1. System Strangling (aka "Strangler Fig"):

- Use Case: Replacing an entire service or a large, vertical slice of a monolith.
- Analogy: A new vine wrapping an old tree until the tree is gone.

2. Branch by Abstraction:

- Use Case: Replacing a component or module inside a single codebase.
- Analogy: Rewiring a single circuit in the house while the main power stays on.

Refactoring techniques

Macro-Pattern 1: System strangling

- Safely and incrementally migrate traffic from an old system to a new one.
- The Key Component
 - The "Proxy" or "Façade." This is not optional.
- This is your "receptionist" that sits in front of everything.
 - It's an API Gateway, Nginx, or just a simple router.



Refactoring techniques

Macro-Pattern 1: System strangling – steps

1. Create the new interface/service
 - (e.g., new-billing-api.com).
2. Insert the Proxy.
 - All traffic (/api/billing/*) now hits the proxy first.
3. Configure the Proxy to pass 100% of traffic to the old system.
 - You've done nothing but add a hop... for now.
4. Implement one endpoint in the new service
 - (e.g., GET /api/billing/invoices).
5. Change the Proxy logic:
 - `if (request.path == "/api/billing/invoices") { route_to_new_service; } else { route_to_old_system; }`
6. Monitor.
 - Repeat for every single endpoint until the old system receives 0% of traffic.

Refactoring techniques

Macro-Pattern 1: System strangling – data problem

- Code is easy; data is hard. **How do the two systems co-exist?**
- Option 1 (fast & dangerous)
 - Shared DB. Both systems read/write to the same database.
 - The new service "strangles" the old tables. High risk of corruption.
- Option 2 (slow & safe):
 - Syncing. New service has its own clean DB.
 - Write "sync" logic (event triggers, nightly jobs) to move data from old to new.
- Option 3 (the goal): “Data Strangulation”
 - The new service becomes the "Source of Truth" for invoices.
 - The old system must now call the new service's API to get invoice data. This completes the strangulation.

Refactoring techniques

Macro-Pattern 2: Branch By Abstraction

- Branch by Abstraction (BBA)
 - This is not git branch. This is a technique to avoid long-lived feature branches.
- The Goal: Replace a component inside a monolith on the main branch without breaking anything.
- Introduces an abstraction layer (like an interface) that "hides" both the old and new implementations.
 - A **Feature Flag** will decide which one gets used.

Refactoring techniques

Macro-Pattern 2: Branch By Abstraction

- Imagine you need to replace “Old Messy Billing Engine”.
- The "Old Way" (Git Branch):
 - You create a branch: `git checkout -b new-billing`.
 - You work for 3 months.
 - Meanwhile, `master` has 5,000 new commits.
 - Dozens of people have patched bugs in the old billing engine.
 - You finally try to merge.
 - You face Merge Hell. It's a week-long, high-risk nightmare of resolving conflicts, and your code (which worked on your branch) is now broken by changes from main.

Refactoring techniques

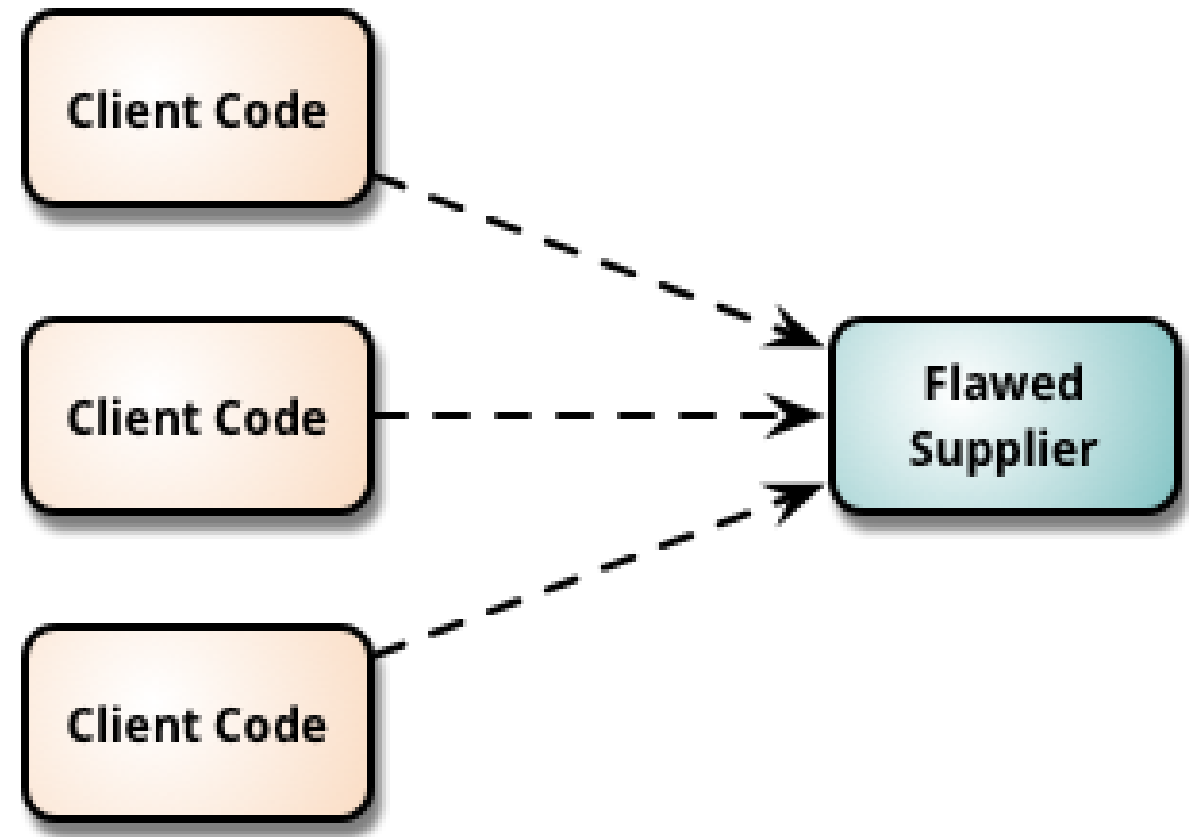
Macro-Pattern 2: Branch By Abstraction

- You stay on “master”.
- You follow the BBA algorithm (below).
 - Your new code (New Clean Billing Engine) is checked into master (but is "dark") on Day 1.
- You and everyone else are integrating against the same master branch, every single day.
 - Using pull requests...
- When you're done, “release” is just flipping a switch.
 - There is zero merge conflict.

Refactoring techniques

Macro-Pattern 2: Branch By Abstraction

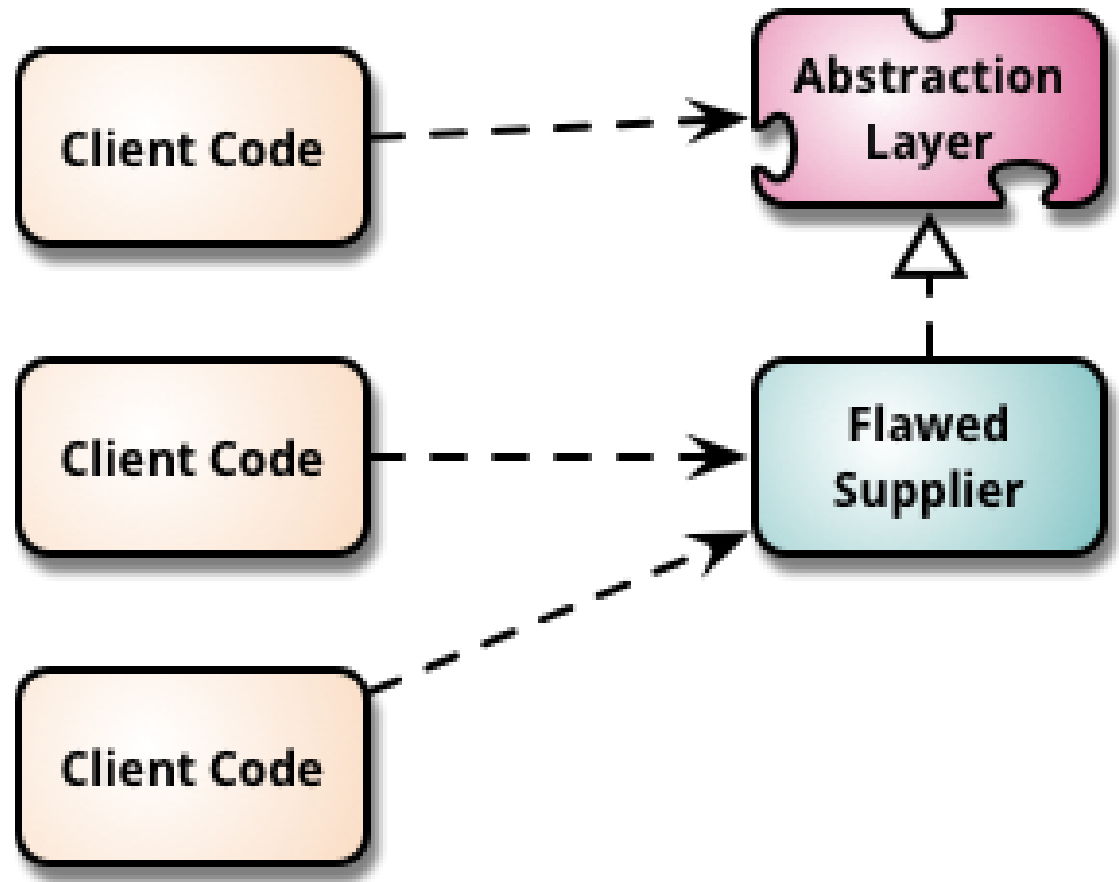
- Several components of the software system rely on a specific module, library, or framework.
- That dependency has become outdated with our future architecture.
- We aim to replace this dependency without disrupting the system's functionality or stability.



Refactoring techniques

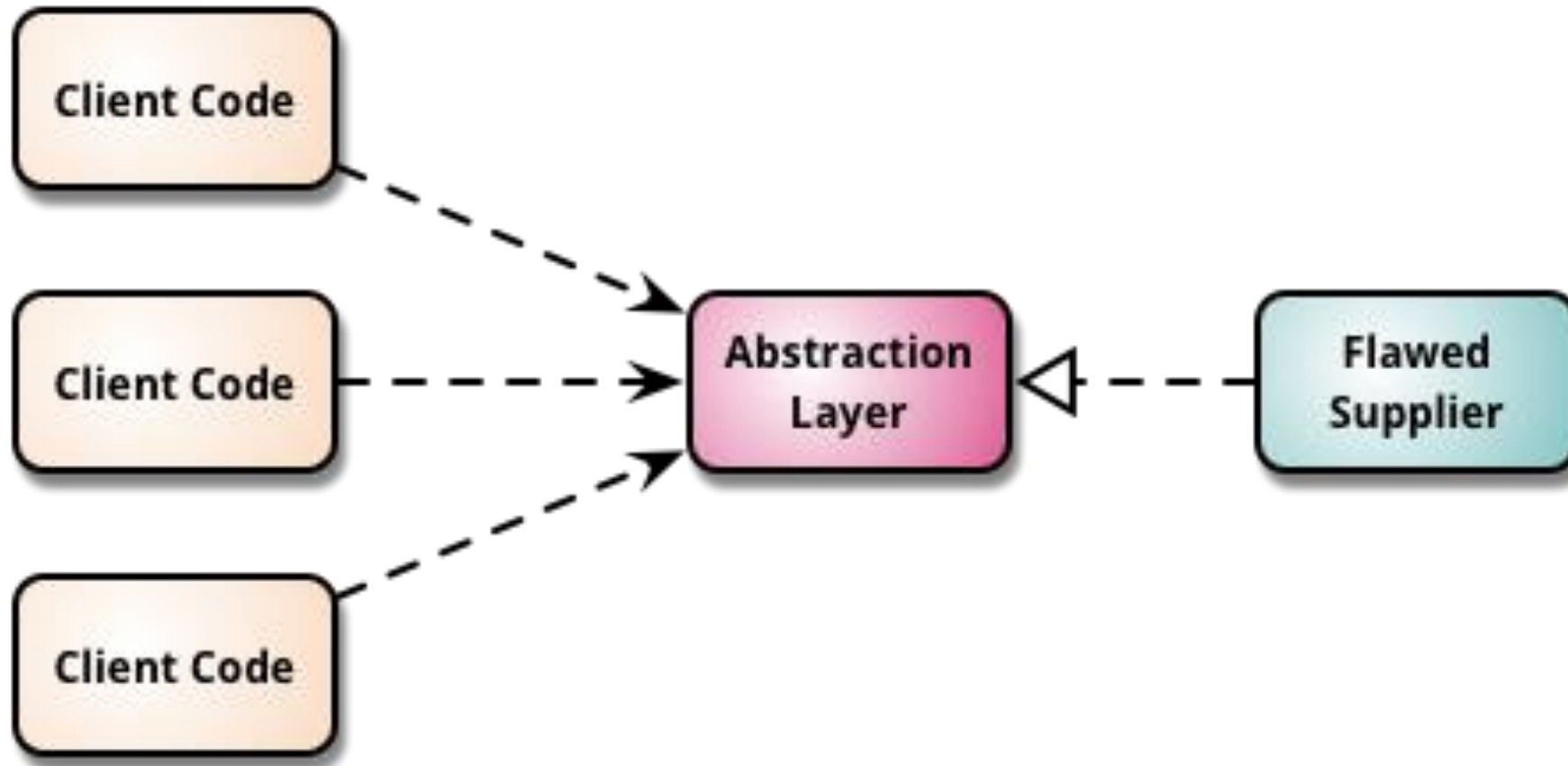
Macro-Pattern 2: Branch By Abstraction

- We create an **abstraction layer**
 - It encapsulates all communication between the **client code** and the **current supplier**.
- Ensure that the client code **no longer depends directly** on the old supplier.
- Future changes to the supplier can be handled behind this layer, without affecting the client.



Refactoring techniques

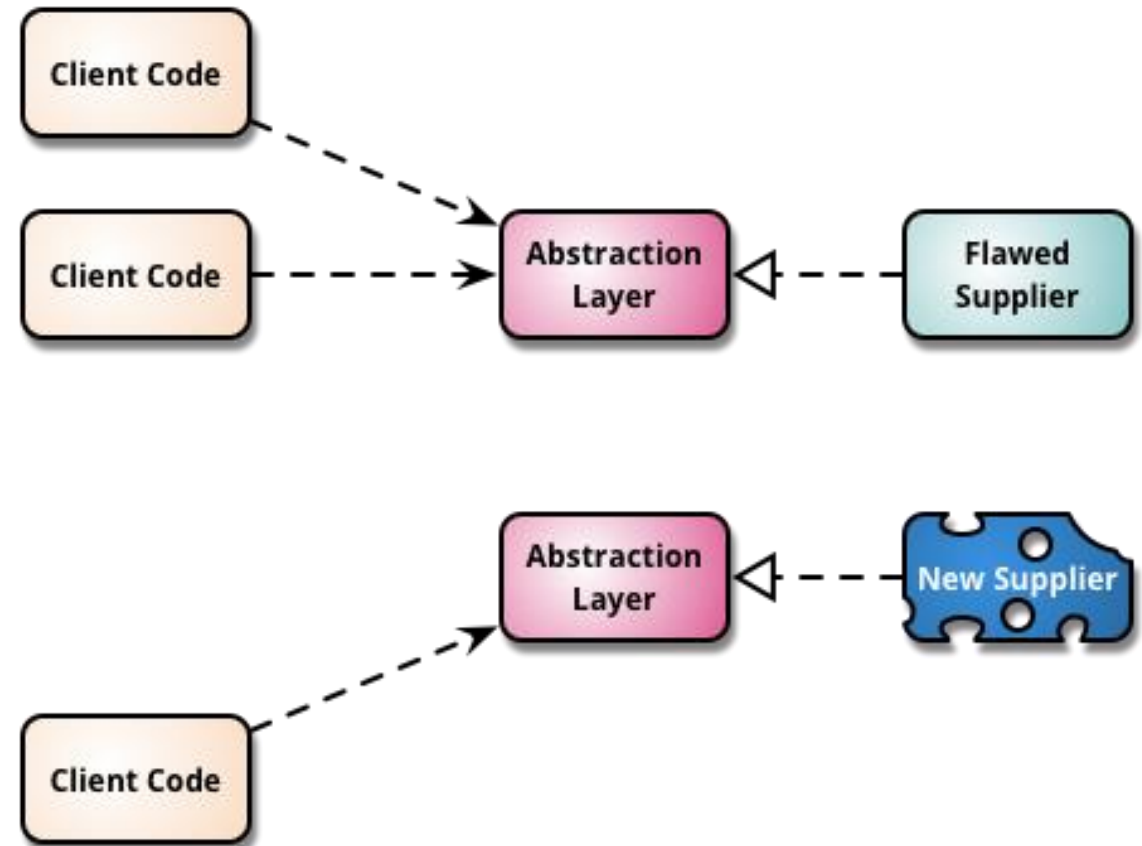
Macro-Pattern 2: Branch By Abstraction



Refactoring techniques

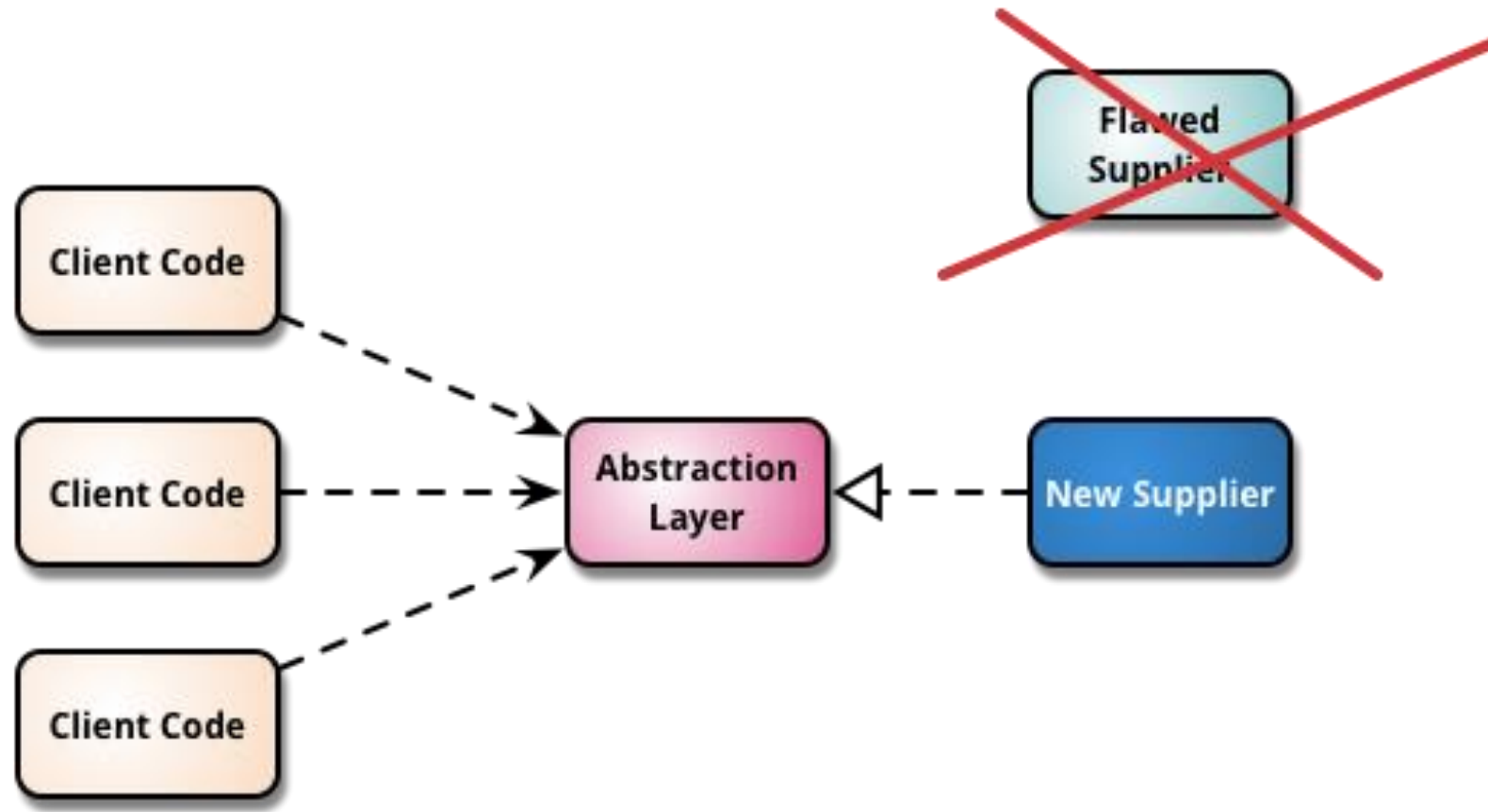
Macro-Pattern 2: Branch By Abstraction

- We develop a new supplier
 - It provides the required functionality for a specific section of the client code
 - Using the same abstraction layer as before.



Refactoring techniques

Macro-Pattern 2: Branch By Abstraction



Nine practices to extend the life of software

1. Say What, Why, and for Whom Before How

- The "How" is the code → the technical implementation.
 - Writing code, changing schemas, designing APIs...
 - In a legacy system, touching the "How" is always expensive and dangerous.

- Forbidden to even think about the "How" until you have crystal-clear, written-down answers to the first three questions.
 - **What** is the specific, observable outcome we need?
 - **Why** are we doing this? What is the business value?
 - **Who** is the specific user this is for?

Nine practices to extend the life of software

1. Say What, Why, and for Whom Before How

➤ “What” (The Problem)

- This is not "build a billing module." It's "the user must be able to download a PDF of their invoice."
- It defines the *exact* requirement, free of any technical jargon.

➤ “Why” (The Justification)

- This is the most important question. If the "Why" is weak, you shouldn't be touching the legacy system at all.
- **Good Why:** "We need to do this because we are legally required to show new tax information, and we are currently non-compliant."
- **Bad Why:** "Because this old code looks ugly."

➤ “For Whom” (The User)

- This defines the scope. "For Whom" tells you what not to build.
- Example: If the feature is "for Customer Service Reps," it has different requirements (speed, data density) than if it's "for first-time mobile users" (simplicity, guidance).

Nine practices to extend the life of software

2. Build in Small Batches

- This principle is about minimizing risk and getting feedback fast.
- A “small batch” means making the smallest possible change that delivers a single, verifiable piece of value.
 - In a legacy system, every change is a gamble.
 - The bigger the change, the bigger the gamble.
- “Building in small batches” is a strategy to stop gambling and start making small, safe bets.

Nine practices to extend the life of software

3. Integrate Continuously

- This principle is the technical engine that makes “2. Building in Small Batches” possible and safe.
- “Integrate Continuously” means merging your code changes into the master shared codebase very frequently.
- This sounds insane, right? Your next question is obvious...
- **"But My Feature Isn't Finished!"**
 - This is the most important part. How can you commit unfinished work to master?

Nine practices to extend the life of software

3. Integrate Continuously

- You don't use *branches* to hide unfinished work.
- You use **Feature Flags** or **Branch by Abstraction**.
- The local "branch" should not live for more than a few hours.
- The master branch is always stable and releasable.
- Everyone commits their work directly to master multiple times per day.

Nine practices to extend the life of software

4. Collaborate

- This principle argues that in a legacy system, **no one is allowed to be a "hero."**
 - The enemy in legacy code is the "knowledge silo."
 - This creates a huge risk (the "bus factor").
 - "Collaborate" is the practice of actively breaking down those silos.

- This isn't just "have meetings." It's about specific, hands-on techniques:
 - **Pair Programming:** This is the most effective way. Two engineers work at one keyboard.
 - **Mob Programming:** The whole team (or a small group) works on one problem, on one computer, at the same time.
 - **Collective Code Ownership:** The team, not an individual, owns every module.

Nine practices to extend the life of software

5. Create CLEAN Code

- This principle is the "Boy Scout Rule" applied to software
 - "Always leave the code cleaner than you found it."
- This principle **does NOT** mean you must rewrite the entire 5,000-line "God" class you're about to touch.
- "Create CLEAN Code" means:
 - Stop the Rot: First, do no harm. Any new code you add must be clean.
 - Make Incremental Improvements: While you're in that messy file to fix a bug, you apply the Boy Scout Rule.
 - You don't clean the whole file, but you make one or two tiny improvements in the immediate area.

Nine practices to extend the life of software

6. Write the Test First

- This principle is the *engine* of safe refactoring.
- In a "greenfield" (new) project, "Write the Test First" means Test-Driven Development (TDD)
 - Write a failing test, make it pass, then refactor.
- In a legacy system, this practice is more nuanced but even more critical.
 - It means: Before you change any existing code, you must first write a test that proves the code works as it does right now.
- Capture the Behavior:
 - You run the test and capture its exact output (a log, a file, a JSON string, etc.), bugs and all. This output is your "Golden Master."

Nine practices to extend the life of software

7. Specify Behaviors with Tests

- This principle argues that tests are the only trustworthy documentation for a legacy system.
- This is a specific form of Behavior-Driven Development (BDD).
 - It moves beyond just testing code and uses the tests themselves to specify what the system is supposed to do in plain, human-readable language.
- Instead of a test named `testUserLogin_InvalidPassword()`, you write a specification:
 - Given** a user exists **And** they are on the login page
 - When** they enter a valid username **And** an invalid password
 - Then** they should see an "Invalid password" error message

Nine practices to extend the life of software

8. Implement the Design Last

- It is the heart of Test-Driven Development (TDD) and is the direct opposite of "Big Design Up Front."
 - It argues that in a complex system, you cannot know the correct design until you've solved the problem.
 - Any "perfect" design you create in a diagram before writing code is based on assumptions that will almost certainly be proven wrong the moment you touch the legacy codebase.

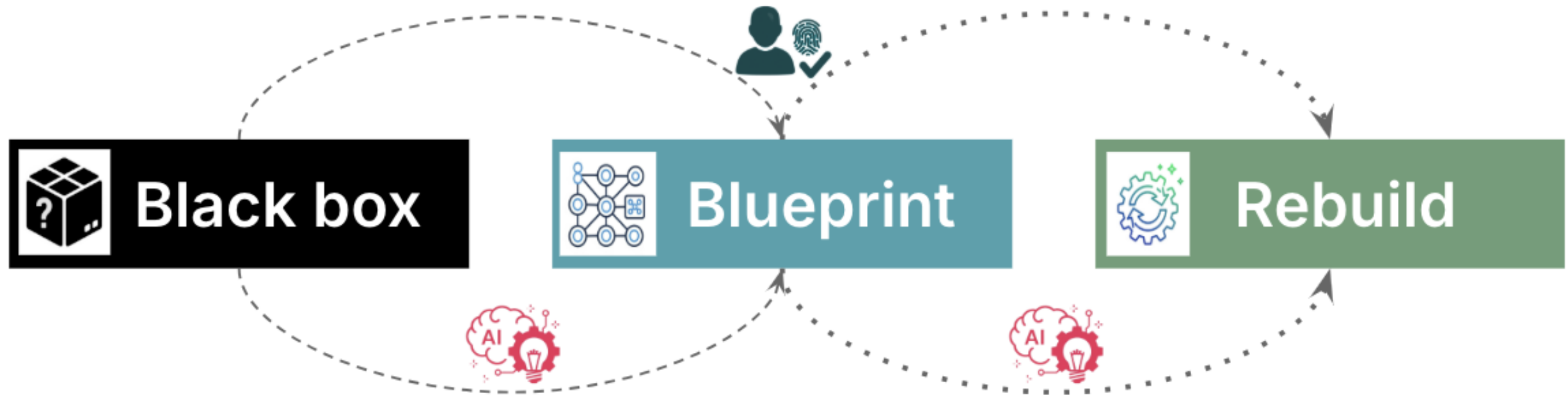
- Implement the Design Last" flips the process:
 - **Write a Test (Practice 7):** First, you write a test that specifies a behavior
 - **Make It Pass (Ugly):** You write the absolute simplest, ugliest, most brute-force code possible to make that test pass.
 - **Refactor (Implement the Design):** Now that the test is green, you have a safety net. The behavior is locked in. This is when you implement the design.

Nine practices to extend the life of software

9. Refactor Legacy Code

- This is the "payoff" practice, the one that all the previous eight practices are designed to enable.
- *"A change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior."*
- In simple terms: Cleaning the code without breaking it.

Black boxes to modern blueprints



Black boxes to modern blueprints

Problem

- This is the "Analysis Paralysis" scenario.
- The system is critical (payroll, logistics) but completely opaque.
- We are facing a Modernization Deadlock:
 - Too risky to replace (you don't know what it does).
 - Too costly to maintain (old OS, security risks).
 - Too critical to ignore.

Black boxes to modern blueprints

Overview

- We cannot recover the original code (and we don't want to).
- We must recover the functional specification (the "what," not the "how").
- Old Way: Years of manual analysis by a team of 20 (as the article notes).
- New Strategy: AI-Assisted Reverse Engineering.
- Use LLMs/GenAI to accelerate turning artifacts into a blueprint.

Black boxes to modern blueprints

The "Multi-Lens" Approach

- We can't trust one source.
 - We must triangulate all available artifacts.
- Lens 1: UI Layer (The "Last Mile")
 - Artifacts: ASP files, JS files, screenshots.
 - What it tells us: User workflows, field names, client-side validation.
- Lens 2: Data Layer (The "State")
 - Artifacts: DB Schema (650+ tables), Stored Procedures (1200+).
 - What it tells us: The business entities, the rules encoded in SQL.
- Lens 3: Opaque App Tier (The "Missing Logic")
 - Artifacts: Compiled binaries (.dll, .exe). No source code.
 - This is the core of the black box.

Black boxes to modern blueprints

Step 1: Start Where Visibility is Highest

- Principle: Start with what you can see. Anchor in artifacts users understand.
- UI Layer Reconstruction:
 - Feed AI the ASP/JS files.
 - Prompt: *"Summarize the UI elements, validation rules, and navigation paths from these files."*
- Data Layer Reconstruction:
 - Feed AI the schema and stored procs.
 - Prompt: *"Analyze this schema. Infer the relationships between these tables. Summarize the business logic in this stored procedure."*
- Key: Preserve Lineage! Track every insight back to its source artifact to prevent hallucination.

Black boxes to modern blueprints

Step 2: Connect the Dots (CDC & Inference)

- Change Data Capture (CDC): If possible, run the app and log database changes.
- This connects the UI to the Data.
 - Example: "When user clicks 'Submit' on screen_ABC... -> ...rows are inserted into TABLE_X and TABLE_Y."
- Server Logic Inference:
 - We now have a hypothesis.
 - The "black box" DLLs must contain the logic that links **screen_ABC** to **TABLE_X**.
- This gives us a target for the hardest part.

Black boxes to modern blueprints

Step 3: AI-Assisted Binary Archaeology

- This is the deep, technical work.
 - We have 45 DLLs with 1000s of functions.
- The Problem: Brute-force (feeding all ASM to an LLM) fails.
 - Context windows overflow, and the AI hallucinates.
- The Solution: Slice the DLL based on functional area (the "subtree" approach).
- We don't need to understand all 4,000 functions.
 - We need the 40+ relevant to our workflow.

Black boxes to modern blueprints

Step 4: Triangulate and Validate (Human-in-the-Loop)

- AI is an accelerator, not an oracle.
 - You must validate its hypotheses.

- Triangulate:
 - Does the function signature from the Binary match the API call from the UI Layer?
 - Does the business logic in the Pseudocode match the DB changes from CDC?

- Human Validation: The final, generated spec must be walked through with a sys-admin or long-time user to confirm the functionality.

Black boxes to modern blueprints

Summary

- Start Where Visibility is Highest: (UI, DB). Don't start with binaries.
- Enrich in Passes: Small chunks. Build context. Don't boil the ocean.
- Triangulate Everything: Never trust one source. (UI + DB + Binary).
- Preserve Lineage: Track every insight to its source artifact. This is your anti-hallucination shield.
- Keep Humans in the Loop: AI accelerates analysis. Humans provide domain understanding.

GenAI in Software Engineering

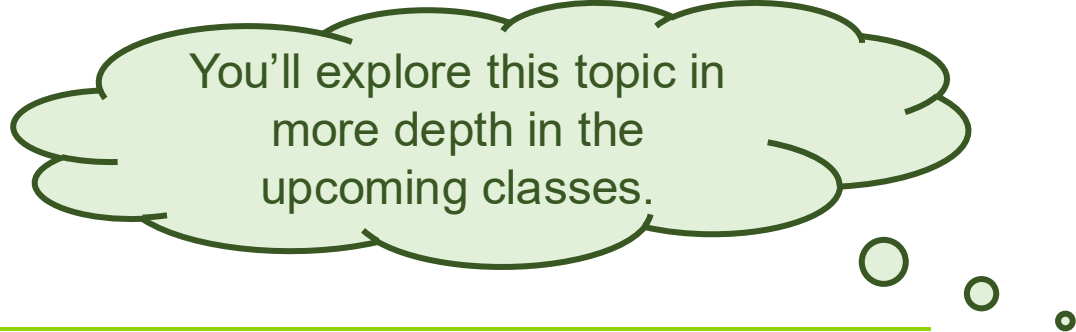
GenAI for Requirements Extraction

- Static & dynamic analysis enhanced by LLM explanations
- Generate readable flowcharts and CRUD maps
- Extract business rules (e.g. COBOL BREX)
- Create documentation usable by non-technical roles

GenAI in Software Engineering

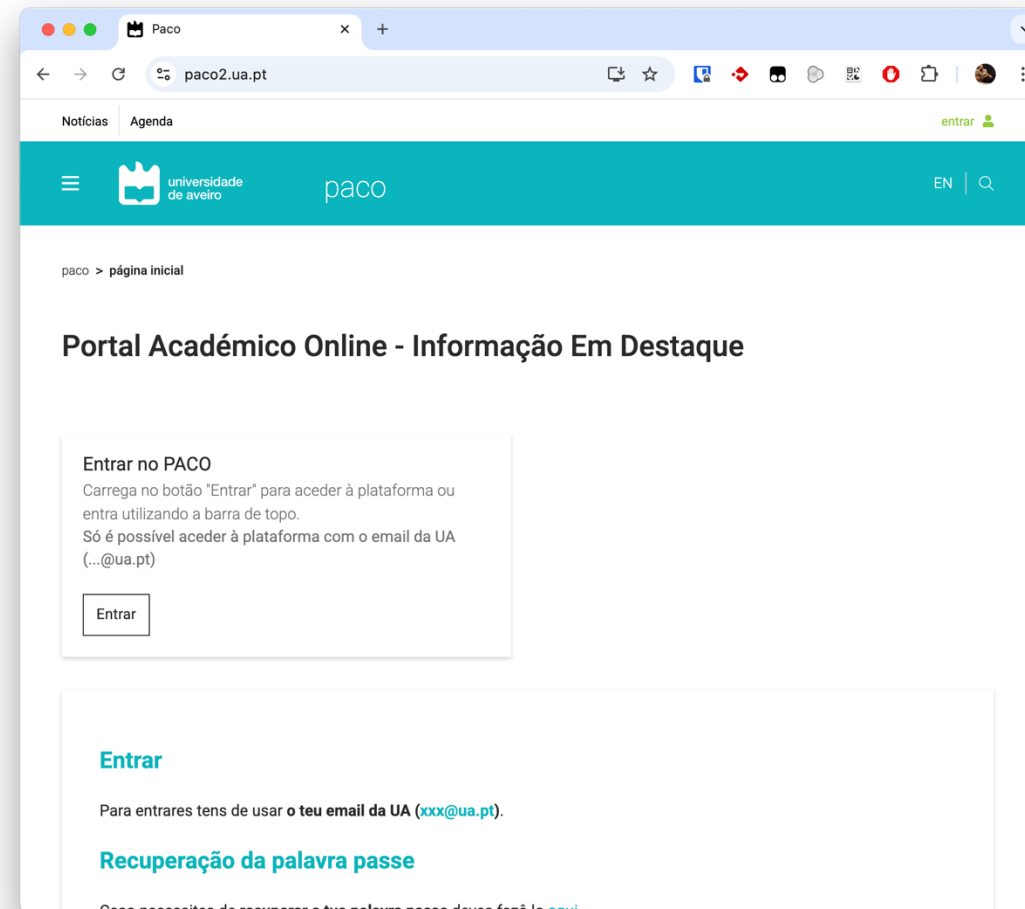
GenAI for Requirements Extraction

- Build *mental models* and *domain maps*
- Use Event Storming + GenAI summaries
- Combine code + documentation in graph
- Enrich chatbot context to answer SME-level questions

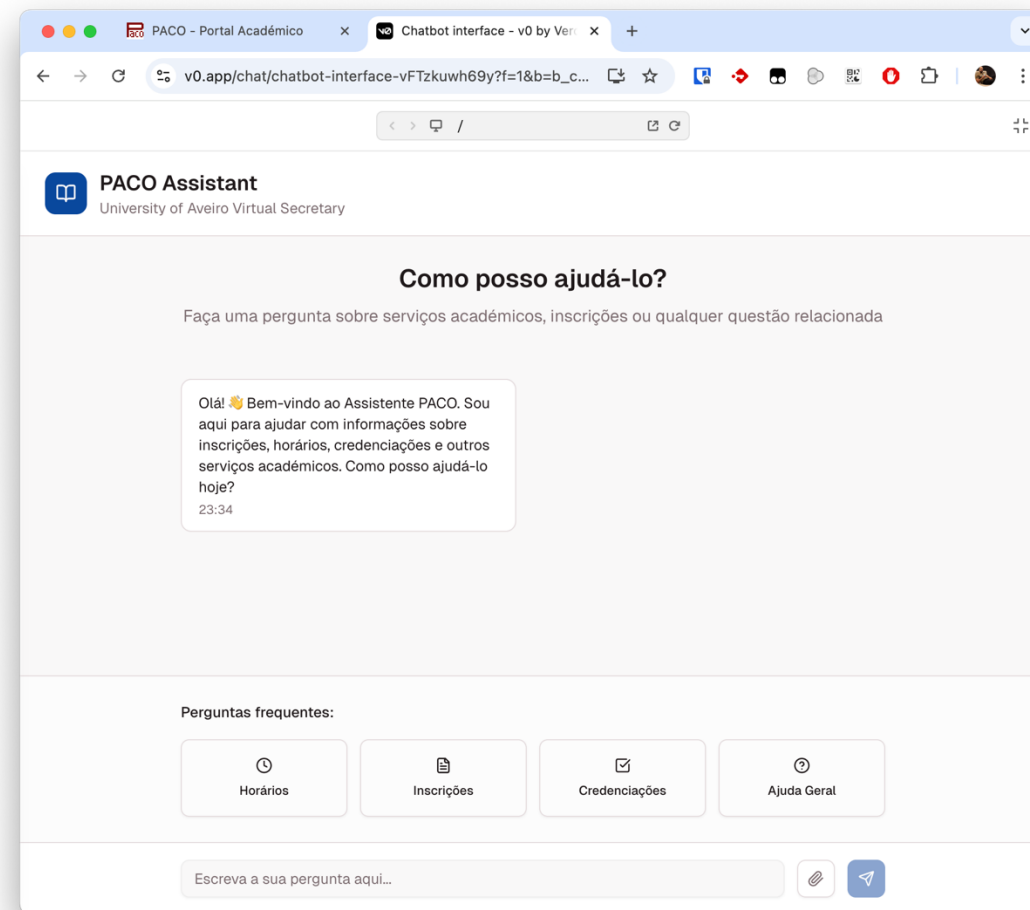
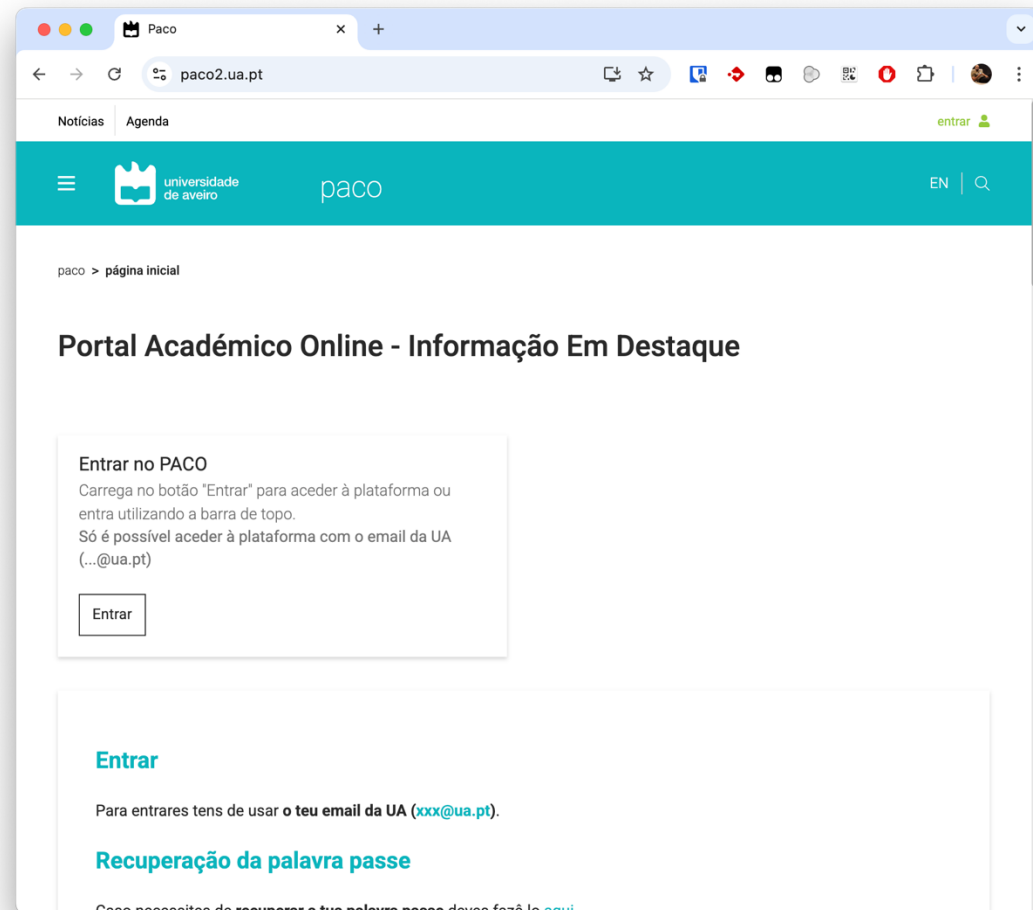


You'll explore this topic in more depth in the upcoming classes.

The present



The future



Project goal

- A new investor loved your application — but wants it aligned with the latest trends.
- Your tasks:
 - Create a conversational assistant to integrate into your tool.
 - Improve observability to consider frontend actions.
- You're free to define the strategy, design, and implementation approach.